CHAPTER 1

The M Word

Metaprogramming is writing code that writes code.

We'll get to a more precise definition soon, but this one will do for now. What do I mean by "code that writes code," and how is that useful in your daily work? Before I answer those questions, let's take a step back and look at programming languages themselves.

Ghost Towns and Marketplaces

Think of your source code as a world teeming with vibrant citizens: variables, classes, methods, and so on. If you want to get technical, you can call these citizens *language constructs*.

In many programming languages, language constructs behave more like ghosts than fleshed-out citizens: you can see them in your source code, but they disappear before the program runs. Take C++, for example. Once the compiler has finished its job, things like *variables* and *methods* have lost their concreteness; they are just locations in memory. You can't ask a class for its instance methods, because by the time you ask the question, the class has faded away. In languages such as C++, runtime is an eerily quiet place—a ghost town.

In other languages, such as Ruby, runtime is more like a busy marketplace. Most language constructs are still there, buzzing all around. You can even walk up to a language construct and ask it questions about itself. This is called *introspection*.

Let's watch introspection in action. Take a look at the following code.